

Perfect 2048: the fewest moves, the highest score, and what the undo button buys

Dom the Developer
<https://github.com/DomTheDeveloper/2048-undo>

Draft, September 2026

Abstract

We study 2048 under *possible play*: the spawn sequence is treated as part of the plan, which is exactly what a player with a re-randomising undo button can realise, and exactly what “possible with positive probability” means for the ordinary game. Three results. (1) On any board, the tile 2^n cannot appear before move $(2^n - 8)/4 + (n - 2)$; the bound is attained, giving 519 for the 2048 tile (recovering Lees-Miller’s Markov-chain figure) and 32,781 for the largest tile 131072 on the 4×4 board, and 65,533 moves for the complete descending chain $131072, 65536, \dots, 4$. (2) On any board with c cells the score is at most $\Phi_c - 4c$ where $\Phi_c = \sum_{k=2}^{c+1} (k-1)2^k$: reaching the top position needs at least one spawned 4 for every one of its c tiles, and each spawned 4 forfeits exactly four points. The proof is a register-allocation (pebbling) argument. For 4×4 this is 3,932,100. Exhaustive enumeration shows the bound is tight on every board with at most nine cells— 2×2 , 2×3 , 2×4 , 3×3 —and reproduces, to the digit, the 48,713,519 positions of 3×3 reported by Yamashita, Kaneko and Nakayashiki. For 4×4 we give an explicit line scoring 3,925,224 in 129,333 moves; whether the remaining 6,876 points can be collected is open, and we conjecture they can. (3) The perfect lines are shipped as data and replayed in a browser; with honest 90/10 spawns the undo button must be pressed about 1.94 million times to play the 32,781-move line, a number we compute exactly. We also strongly solve the boards up to nine cells (optimal expected score, tile probabilities) and benchmark an expectimax player against the classical heuristics on the full board.

1 Introduction

2048 [2] is played on a 4×4 board. A move slides all tiles in one of four directions; equal tiles that collide merge into their sum, each tile merging at most once per move; then one new tile appears in a uniformly random empty cell, a 2 with probability 0.9 and a 4 with probability 0.1. The score is the sum of all merged values. The game ends when no move changes the board.

Alok Menghrajani’s fork [18] adds an undo button that re-randomises the spawn on replay. That small change turns the game into a different object: a player who is willing to press undo often enough can realise *any* spawn sequence, and so can play any game that is possible in principle. This paper is about those games. Three questions organise it:

1. What is the fewest moves in which the largest tile can be built?
2. What is the highest score any game can reach?
3. What does the undo button cost, in presses, to play such a game?

The first two questions have folklore answers. For the move count, the observation that every move adds a spawned 2 or 4 to the board gives a lower bound immediately; the cascade that finishes the tile adds a few more moves, and it was not obvious that the sum is exact. For the score, several numbers circulate: 3,932,164 (pure mass accounting), 3,932,156 (two “structurally forced” 4-spawns), and 3,932,100 (one forced 4-spawn per tile of the final position). Our own earlier README argued for a ceiling of 3,932,156 and claimed geometry prevents paying it. That claim was half right: geometry does prevent it, but the right ceiling is 3,932,100, and we prove it.

Contributions.

- Theorem 4: a complete proof that 2^n needs at least $(2^n - 8)/4 + (n - 2)$ moves on any board, with equality for $n = 11$ (519) and $n = 17$ (32,781) on 4×4 , and for every tile on every board of at most nine cells.
- Theorem 8: on any c -cell board, score $\leq \Phi_c - 4c$. The argument identifies a merge tree’s cell requirement with its Strahler number (the Ershov register count) and shows that the c final tiles must pay their tight moments in a forced order.
- Exact values (Section 5.3): the bound is attained on 2×2 , 2×3 , 2×4 and 3×3 ; the enumerator agrees with the published 3×3 state count.
- Section 5.4: for 4×4 , an explicit 129,333-move line scoring 3,925,224 with 1,735 spawned 4s, where the theorem allows 16; we locate where the construction spends its 4s and conjecture the ceiling is attainable.
- Section 6: the exact expected number of undo presses for each shipped line under honest odds, and the opening problem.
- Section 7: a strong solution of the small boards (expected score, tile probabilities) and an expectimax player for the full board measured against the classical heuristics and an adversarial tile generator.

2 Related work

Complexity and combinatorics. Langerman and Uno [10] showed that deciding whether a given position can be played to a given tile is NP-hard on $m \times n$ boards for Threes, 1024 and 2048; Abdelkader, Acharya and Dasler [1] showed the variant without new tiles is still hard; Mehta [17] gave a PSPACE-completeness result for a related decision problem. Eppstein [3] analysed the maximum score in a relaxed model of 2048-like games in which any set of cells whose values sum to a tile value may be merged in one step, proving a min-max theorem that equates the maximum score with the smallest value for which greedy change-making needs a given number of coins; the binary-counter strategy every strong 2048 player follows is, in that light, greedy change-making in base 2. The relaxed model has only the small spawn value. The real game’s 4-spawns are precisely the resource that buys an extra cell, and Theorem 8 prices that resource: four points each, and exactly c of them on the way to the top position.

Learning agents. Szubert and Jaśkowski [24] introduced temporal-difference learning of n -tuple networks for 2048; Wu et al. [26] added multi-stage learning; Jaśkowski [8] reached the strongest reported agents; Matsuzaki and co-authors studied the systematic selection of n -tuples and neural-network players [19, 14, 15, 16]; Guei, Wei and Wu [7, 6] used 2048-like games for teaching reinforcement learning and proposed optimistic TD learning. Slizkov [23] proved rigorous bounds on honest play: a strategy reaching 2048 with probability at least 0.99969, and 256 with certainty.

Exact solving. Lees-Miller [12, 11, 13] analysed the game with Markov chains and Markov decision processes, computed the minimum of 519 moves to the 2048 tile, and enumerated the 2×2 and 3×3 games. Yamashita, Kaneko and Nakayashiki [28] strongly solved 3×3 (48,713,519 positions) and Kaneko and Yamashita [9] strongly solved 4×3 (1,152,817,492,752 positions, 739,648,886,170 afterstates, optimal expected score about 50,724.26 from a two-2s start), the key technique being to partition the state space by the sum of tile values (the “age” of a state). Our enumerator uses the same grading and, on 3×3 , reproduces their position count exactly (Section 5.3).

3 Preliminaries

A board is a finite set of c cells with a slide structure (rows and columns in the rectangular case; the arguments below use only the cell count). Tiles carry values 2^k , $k \geq 1$; we call k the *rank*. A *play* is a sequence of moves, each consisting of a slide (which performs its merges) followed by a *spawn* of a 2 or a 4 into an empty cell. The game starts with two spawns. Let n_2 and n_4 be the numbers of spawned 2s and 4s so far, counting the two starting tiles.

Definition 1 (possible play). In *possible play* the spawn values and cells are chosen by the player. Every finite spawn sequence has positive probability in the honest game, so a position is reachable in possible play if and only if it is reachable with positive probability in the honest game; and a player with a re-randomising undo button can realise any possible play.

For a position B write $\text{mass}(B) = \sum_{t \in B} 2^{k_t}$ and

$$\Phi(B) = \sum_{t \in B} (k_t - 1) 2^{k_t}.$$

Lemma 2 (mass ledger). *Slides conserve mass and every spawn adds its value: $\text{mass}(B) = 2n_2 + 4n_4$ at all times. After m moves the mass is at most $8 + 4m$, with equality if and only if every spawn so far, the two starting tiles included, was a 4.*

Lemma 3 (score identity). *At every moment of every play, $\text{score} = \Phi(B) - 4n_4$.*

Proof. By induction over events. Initially the score is 0; a starting 2 contributes 0 to Φ , a starting 4 contributes 4 and raises $4n_4$ by 4. A merge of two 2^k tiles into 2^{k+1} scores 2^{k+1} and changes Φ by $k2^{k+1} - 2(k-1)2^k = 2^{k+1}$. A spawned 2 changes nothing on either side; a spawned 4 adds 4 to Φ and 4 to $4n_4$. $\square$

So every spawned 4 costs exactly four points relative to what the final position “looks like it is worth”, and the maximum score of a game is $\max_B (\Phi(B) - 4n_4^{\min}(B))$ over reachable positions B , where $n_4^{\min}(B)$ is the fewest spawned 4s on any play reaching B .

Merge trees. Every tile on the board is the root of a binary *merge tree*: its leaves are spawned tiles (value 2 or 4), each internal node is a tile created by merging its two children. A tile of rank k has all leaves at depth $k-1$ (2-leaves) or $k-2$ (4-leaves). Two facts about the game drive everything below:

- *Persistence.* A tile, once spawned or created, stays on the board until it merges into its parent. Nothing else removes it.
- *One doubling per move.* A tile’s value at most doubles in a move, because each tile merges at most once per move.

Call the tiles present on the board that belong to the merge tree of a future tile T the *pieces* of T . By persistence, from the first spawn of a leaf of T until T is created, at least one piece of T is on the board; afterwards T itself is.

4 The fewest moves

Theorem 4. *On any board and for any play, the tile 2^n ($n \geq 3$) does not exist before move*

$$M(n) = \frac{2^n - 8}{4} + (n - 2).$$

Equality is attainable only if the game starts from two 4s and every spawn outside the last $n - 2$ moves is a 4.

Proof. Let T be the first move after which a tile 2^n exists. A tile spawned at move $T - j$ has value at most $4 = 2^2$ then, and by one doubling per move its value at move T is at most 2^{2+j} ; for it to be part of the 2^n we need $2 + j \geq n$, i.e. $j \geq n - 2$. Hence the $n - 2$ tiles spawned at moves $T - (n - 3), \dots, T$ are not part of the 2^n : they are still on the board (persistence), as themselves or merged among each other, so the position after move T has mass at least 2^n plus the sum of those $n - 2$ spawn values.

Write z for the number of spawned 2s among the T move-spawns and z' for the number among the last $n - 2$ of them, so $z' \leq z$. By Lemma 2 the mass after move T is $M_0 + 4T - 2z$ with $M_0 \leq 8$ the starting mass, and the last $n - 2$ spawns sum to $4(n - 2) - 2z'$. Therefore

$$M_0 + 4T - 2z \geq 2^n + 4(n - 2) - 2z', \quad \text{so} \quad 4T \geq 2^n - 8 + 4(n - 2) + 2(z - z') + (8 - M_0) \geq 2^n - 8 + 4(n - 2),$$

which is the claim. Equality forces $M_0 = 8$ and $z = z'$. $\square$

Attainability is a construction. For $n = 11$, Lees-Miller's Markov chain [12] exhibits the $519 = 510 + 9$ move win on 4×4 . For $n = 17$ we generated, verified and ship a 32,781-move line (Section 6.1); Table 1 shows that the bound is attained for every tile on every board up to nine cells.

Corollary 5. *On 4×4 the complete descending chain $131072, 65536, \dots, 4$ (one power of two per cell, a dead position) needs exactly 65,533 moves.*

Proof. Its mass is $2^{18} - 4 = 262,140$; by Lemma 2 at least $(262,140 - 8)/4 = 65,533$ moves are needed and every spawn must be a 4. The shipped line (Section 6.1) does it in exactly that many. Along it the 131072 first forms at move 32,784, three later than $M(17)$: all-4 feeding needs a few consolidations that a 2-spawn during the cascade would avoid, and the ledger absorbs them. $\square$

5 The highest score

5.1 A merge tree needs its Strahler number of cells

The *Strahler number* of a binary tree is $S(\text{leaf}) = 1$ and, for a node with subtrees A, B , $S = \max(SA, SB)$ if they differ and $SA + 1$ if they are equal. It is the number of registers needed to evaluate an expression tree without spilling (Ershov [4], Sethi-Ullman [22], Flajolet-Raoult-Vuillemin [5]) and the black pebbling number of the tree [20]. Building a tile is evaluating its merge tree with cells as registers, and the persistence of pieces makes the lower bound short to prove directly.

Lemma 6. *Let T be a tile of rank $k \geq 2$ with merge tree $\mathcal{T}$. Then $S(\mathcal{T}) = k$ if every leaf is a 2, and $S(\mathcal{T}) = k - 1$ if at least one leaf is a 4.*

Proof. Induction on k . For $k = 2$: a spawned 4 is a leaf ($S = 1$), and $2 + 2$ has $S = 2$. For $k > 2$ the children are trees of rank $k - 1$ with Strahler numbers in $\{k - 2, k - 1\}$, equal to $k - 1$ exactly when all their leaves are 2s. Two all-2 children give $S = k$; otherwise the maximum is $k - 1$ and, if both are $k - 2$, $S = k - 1$ as well. $\square$

Lemma 7 (tight moment). *During the construction of a tile with merge tree $\mathcal{T}$ there is a moment at which at least $S(\mathcal{T})$ pieces of $\mathcal{T}$ are on the board simultaneously.*

Proof. Induction on $\mathcal{T}$; a leaf is its own piece. Let the root have subtrees A and B , roots created at times $t_A < t_B$ (if they are created in the same move, either order works), merged at time $t^* > t_B$. Let τ_A, τ_B be tight moments of A, B given by induction, so $\tau_A \leq t_A$ and $\tau_B \leq t_B$. If $SA > SB$, or $SB > SA$, the larger side's tight moment already shows $\max(SA, SB)$ pieces. Suppose $SA = SB = s$; we need $s + 1$.

- If $\tau_B > t_A$: at τ_B the s pieces of B are on the board and so is the finished root of A (created at $t_A < \tau_B < t_B \leq t^*$, and persistent): $s + 1$ pieces.
- If $\tau_B \leq t_A$ and $\tau_A > t_B$: symmetric, s pieces of A plus the root of B .
- If $\tau_B \leq t_A$ and $\tau_A \leq t_B$: from τ_B until t_B at least one piece of B is on the board (persistence), and $\tau_A \leq t_A < t_B$; if $\tau_A \geq \tau_B$ then at τ_A there are s pieces of A and at least one of B . If instead $\tau_A < \tau_B \leq t_A$, then at τ_B there are s pieces of B and at least one piece of A (present from τ_A until $t_A \geq \tau_B$).

In every case some moment shows $s + 1$ pieces of $\mathcal{T}$. $\square$

5.2 The ceiling

Let $\Phi_c = \sum_{k=2}^{c+1} (k-1)2^k = (c-1)2^{c+2} + 4$, the value of Φ on the *full chain* $2^{c+1}, 2^c, \dots, 2^2$: one distinct power of two per cell.

Theorem 8. *On any board with c cells, every play satisfies $\text{score} \leq \Phi_c - 4c$. Equality requires the final position to be the full chain, reached with exactly one spawned 4 in the merge tree of each of its c tiles.*

Proof. Fix a play and its final position with tiles $T_1, \dots, T_m$, $m \leq c$, of ranks $k_1, \dots, k_m$. Every spawned tile of the whole play is a leaf of exactly one T_i , so $n_4 = \sum_i h'_i$ where h'_i counts the 4-leaves of T_i ; put $h_i = \min(h'_i, 1)$. By Lemma 6 the Strahler number of T_i 's tree is $S_i = k_i - h_i$ (a tile of rank 1 is a spawned 2, $S_i = 1$).

Let τ_i be a tight moment of T_i (Lemma 7) and relabel so that $\tau_1 \leq \tau_2 \leq \dots \leq \tau_m$. At τ_p the board holds S_p pieces of T_p and, for each $q < p$, at least one piece of T_q or T_q itself (persistence, since $\tau_q \leq \tau_p$). Hence $S_p + (p-1) \leq c$, i.e.

$$k_p \leq c + 1 - p + h_p \quad (p = 1, \dots, m).$$

By Lemma 3,

$$\text{score} = \sum_p (k_p - 1)2^{k_p} - 4n_4 \leq \sum_p \left[(k_p - 1)2^{k_p} - 4h_p \right].$$

For fixed p the bracket, subject to $k_p \leq c + 1 - p + h_p$, is largest with $h_p = 1$ and $k_p = c + 2 - p$, worth $(c + 1 - p)2^{c+2-p} - 4$; the alternative $h_p = 0$, $k_p = c + 1 - p$ is worth $(c - p)2^{c+1-p}$, which is

board	cells	positions	(raw)	max score	$\Phi_c - 4c$	$\Phi_c - 8$
2×2	4	110	662	180	180	188
2×3	6	21,752	85,844	1,260	1,260	1,276
2×4	8	4,980,767	19,920,382	7,140	7,140	7,164
3×3	9	48,713,519	388,921,077	16,352	16,352	16,380
4×4	16	?	?	$\geq 3,925,224$	3,932,100	3,932,156

Table 1: Exact maximum scores in possible play. “positions” counts reachable positions up to the board’s symmetries (8 for squares, 4 for rectangles), “raw” without reduction. On every board up to nine cells the maximum equals the ceiling of Theorem 8, and the full chain is reached with exactly c spawned 4s. The 3×3 count agrees with Yamashita et al. [28, 9].

smaller for $p < c$ and equal ($= 0$) for $p = c$. All the terms are nonnegative, so filling the board ($m = c$) is best, and

$$\text{score} \leq \sum_{p=1}^c \left[(c+1-p)2^{c+2-p} - 4 \right] = \sum_{k=2}^{c+1} (k-1)2^k - 4c = \Phi_c - 4c.$$

Equality forces $k_p = c+2-p$ and $h_p = 1$ with $h'_p = 1$ for every p . $\square$

Corollary 9. *On 4×4 , $\text{score} \leq \Phi_{16} - 64 = 3,932,164 - 64 = 3,932,100$. The numbers 3,932,164 and 3,932,156 are not attainable.*

Remark 10. The theorem uses only the number of cells, persistence, and one merge per tile per move. It holds for any adjacency structure, any spawn rule that offers the values 2 and 4, and in particular in possible play. The constraint that actually binds is that a tile of rank k built from 2s alone needs k cells at once, and the c tiles of the top position must take their tight moments in decreasing order of size on a board that the larger ones are already sitting on.

5.3 Exact values on small boards

The mass of a position grows by exactly the spawned value each move, so the state graph is a directed acyclic graph graded by mass, and can be enumerated one layer at a time with only the two open layers in memory. Kaneko and Yamashita [9] call this grading the age of a state. We implemented it (`test/solve.js`) with positions reduced by the board’s symmetry group and, per position, the fewest spawned 4s and the fewest moves on any play reaching it; the maximum score is then $\max_B(\Phi(B) - 4n_4^{\min}(B))$ by Lemma 3. The same layered pass, run backwards, strongly solves the honest game: the optimal expected score and, for each tile, the maximum probability of ever building it.

Table 1 summarises. On every board tested the ceiling is attained, always on the full chain, always with exactly one spawned 4 per tile; the fewest moves to every tile 2^k with $k \geq 3$ equals $M(k)$ of Theorem 4. The 3×3 enumeration gives 48,713,519 positions, the number Yamashita, Kaneko and Nakayashiki report, which also fixes the convention of their count: positions up to the eight symmetries of the square. (Lees-Miller’s “about 25 million” for 3×3 counts differently.) The honest-game side of the same computation is in Table 2.

board	$\mathbb{E}[\text{score}]$, standard start	given a two-2s start	max probability of the largest tiles	
2×2	66.96	67.70	16: 96.25%,	32: 8.29%
2×3	480.26	480.99	64: 93.93%,	128: 5.70%
2×4	2,641.87	2,642.60	256: 86.98%,	512: 2.46%
3×3	5,468.49	5,469.21	512: 73.68%,	1024: 1.13%

Table 2: Strong solutions of the honest game (uniform empty cell, 90% twos). The expected score is under the score-maximising policy; each tile probability is under the policy that maximises that probability. The two-2s column uses Kaneko and Yamashita’s convention (their 4×3 figure is 50,724.26).

5.4 The 4×4 board

The 4×4 state space is out of reach of enumeration (Lees-Miller’s month-long run [11] enumerated $1.3 \cdot 10^{12}$ states without finishing; the full game is far larger). What we have is a constructive lower bound. A generator (`test/gen_perfect.js`) walks a binary counter along the snake path into a corner, feeding 2s, and grants a spawned 4 only when a bounded look-ahead shows every pure-2 continuation dying. It produced a line of 129,333 moves ending on the full chain with $n_4 = 1,735$, hence a score of

$$\Phi_{16} - 4 \cdot 1,735 = 3,932,164 - 6,940 = 3,925,224,$$

99.825% of the ceiling and 6,876 points short of it. The identities $\text{moves} = 131,068 - n_4$ (all 129,333 moves spawn 2 except the 4s; $\text{mass } 262,140 = 8 + 2 \cdot \text{moves} + 2n_4$) and $\text{score} = 3,932,164 - 4n_4$ are verified by replay through the game engine in all four corners and both spiral orientations.

Where do the 1,735 go? The first act—building 131072 on the empty board—spends exactly one, at its tight moment. The whole gap is in the second act, the chain 65536, $\dots$, 4 on the fifteen cells beside the 131072: 295 spawned 4s while building the 65536 and 1,439 below it, most of them with three to five empty cells still on the board. So the generator is not being squeezed by the pebbling bound; it is being squeezed by geometry within a cascade. When the counter overflows, the resulting chain of merges takes one move per level, a tile spawns during each of those moves, and the spawns land in the cells the cascade has just freed, alongside the tail of the chain. The bounded look-ahead cannot always see how to digest that junk with 2s alone and buys its way out with a 4. Theorem 8 says the price of the second act is at least fifteen 4s; the small boards, which contain the same corner turns and the same overflow cascades in miniature, all pay exactly the theorem’s price.

Conjecture 11. *The maximum score in 4×4 2048 is 3,932,100, attained by a play with exactly sixteen spawned 4s.*

Two routes to settle it suggest themselves. The direct route is a better search for the second act: the target is a schedule that, at each level k of the chain, builds the first copy of 2^{k-1} from 2s on the $k - 1$ free cells (which Lemma 6 permits) and the second copy with a single 4 at its own tight moment, and that handles the overflow junk explicitly. The indirect route is Kaneko and Yamashita’s enumeration: their layered pass over 4×3 can carry a two-byte “fewest 4s” field at negligible cost and would settle the 4×3 maximum, which Theorem 8 bounds by $\Phi_{12} - 48 = 180,180$; we predict equality.

6 The undo button

6.1 The shipped lines

Three lines were generated once and are shipped as data (`js/perfect_line.js`, two hex characters per move: direction, spawn cell, spawn value): the 32,781-move line to 131072, the 65,533-move line to the full chain, and the 129,333-move maximum score line. The other three corners and the column-first spirals are the seven symmetries of the square applied to the same data. A replay through the real engine checks that every slide moves, every spawn cell is empty, and the counts land exactly.

6.2 What a perfect game costs in undos

Under honest odds a spawn lands in a required cell with a required value with probability $p = \frac{1}{e} \cdot q$, where e is the number of empty cells after the slide and $q \in \{0.9, 0.1\}$ the value’s probability. With a re-randomising undo, the number of attempts is geometric and the expected number of undos for the move is $1/p - 1$; summing over the line gives the exact expectation. The opening pair is the same problem once: two 4s in two prescribed cells happen with probability $2 \cdot \frac{1}{16} \cdot \frac{1}{15} \cdot 0.01$, one game in 12,000, so the line to 131072 starts with an expected 11,999 restarts.

line	moves	$\mathbb{E}[\text{undos in play}]$	$\mathbb{E}[\text{opening restarts}]$	total
131072 (all 4s)	32,781	1,929,679	11,999	1,941,678
full chain (all 4s)	65,533	3,521,967	11,999	3,533,966
maximum score (2s)	129,333	658,984	147	659,131

Measured runs match: the browser reports 1.91–1.97 million undos for the 131072 line. The maximum-score line is four times longer but three times cheaper in undos, because a 2 is nine times as likely as a 4.

7 Honest play

For the full board we implemented an expectimax player (GENIUS) in the tradition of Xiao’s [27]: rows and columns are table lookups over ranks, the heuristic scores empty cells, available merges, monotonicity and a tile-sum penalty, with a pull toward the chosen corner; depth adapts to the number of distinct tiles. Against Richardson’s [21] Evil generator, which places each new tile on the edge the last move packed against beside the largest neighbours, the search models that rule exactly instead of averaging over luck. We ported Richardson’s four players (Smart, Algorithm, Priority, Random) and the generator to the same array engine for a like-for-like comparison. Ten games each, bottom-right corner:

player	regular tiles	Evil tiles	regular, undo to escape d
GENIUS	16384 ×1, 8192 ×5, 4096 ×3, 1024 ×1; 130,131	4096 ×6, 2048 ×4; 56,850	32768 ×2 (2 gar
Smart	4096 ×3, 2048 ×6, 1024 ×1; 46,029	1024 ×4, 512 ×4, 256 ×2; 8,410	16384 ×3, 8192
Algorithm	512 ×5, 256 ×2, 128 ×3; 4,256	256/128/64; 2,011	1024 ×6, 512
Priority	512 ×1, 256 ×6, 128 ×2, 16 ×1; 2,952	128/64; 1,284	
Random	128 ×6, 64 ×3, 32 ×1; 983	128/64/32; 508	256 ×9, 512

Semicolons separate the largest-tile histogram from the average score. The last column is the undo button used as a human uses it: only to escape game over, each death unwinding a doubling number of moves, giving up after forty deaths without a new best score.

8 Open problems

1. Conjecture 11: a 4×4 play with sixteen spawned 4s.
2. The maximum score of 4×3 (predicted 180,180) and 2×5 , within reach of layered enumeration.
3. Whether Theorem 8 is tight on *every* board, and whether a uniform construction exists.
4. Minimum undos: the expected undo count above is for the shipped lines; the line minimising expected undos (which would prefer spawns with many empty cells) is a different optimisation.

Reproducibility

All code is in the repository [25]. `node test/solve.js 3x3` reproduces Table 1 and Table 2 (3×3 takes about five minutes); `PERFECT=1 node test/run.js` replays each shipped line through the game engine and asserts the exact counts (`GOAL=spiral|score`, `ORIENT=col`); `node test/gen_perfect.js` regenerates a line from nothing; `node test/honest.js` and `node test/honest_drive.js` produce the honest-play table.

References

- [1] Ahmed Abdelkader, Aditya Acharya, and Philip Dasler. 2048 without new tiles is still hard. In *8th International Conference on Fun with Algorithms (FUN 2016)*, volume 49 of *LIPICs*, pages 1:1–1:14, 2016.
- [2] Gabriele Cirulli. 2048. <https://github.com/gabrielecirulli/2048>, 2014.
- [3] David Eppstein. Making change in 2048. In *9th International Conference on Fun with Algorithms (FUN 2018)*, volume 100 of *LIPICs*, pages 21:1–21:13, 2018. doi:10.4230/LIPICs.FUN.2018.21.
- [4] A. P. Ershov. On programming of arithmetic operations. *Communications of the ACM*, 1(8):3–6, 1958.
- [5] Philippe Flajolet, Jean-Claude Raoult, and Jean Vuillemin. The number of registers required for evaluating arithmetic expressions. *Theoretical Computer Science*, 9(1):99–125, 1979.
- [6] Hung Guei, Lung-Pin Chen, and I-Chen Wu. Optimistic temporal difference learning for 2048. arXiv:2111.11090, 2021.
- [7] Hung Guei, Ting-Han Wei, and I-Chen Wu. 2048-like games for teaching reinforcement learning. *ICGA Journal*, 42, 2020. doi:10.3233/ICG-200144.
- [8] Wojciech Jaśkowski. Mastering 2048 with delayed temporal coherence learning, multistage weight promotion, redundant encoding, and carousel shaping. *IEEE Transactions on Games*, 10(1):3–14, 2018.

- [9] Tomoyuki Kaneko and Shuhei Yamashita. Strongly solving 2048 4×3 . *ICGA Journal*, 2026. doi:10.1177/13896911261443437; arXiv:2510.04580.
- [10] Stefan Langerman and Yushi Uno. Threes!, Fives, 1024!, and 2048 are hard. In *8th International Conference on Fun with Algorithms (FUN 2016)*, volume 49 of *LIPICs*, pages 22:1–22:14, 2016.
- [11] John Lees-Miller. The mathematics of 2048: counting states by exhaustive enumeration. <https://jdml.info/articles/2017/12/10/counting-states-enumeration-2048.html>, 2017.
- [12] John Lees-Miller. The mathematics of 2048: minimum moves to win with Markov chains. <https://jdml.info/articles/2017/08/05/markov-chain-2048.html>, 2017.
- [13] John Lees-Miller. The mathematics of 2048: optimal play with Markov decision processes. <https://jdml.info/articles/2018/03/18/markov-decision-process-2048.html>, 2018.
- [14] Kiminori Matsuzaki. Systematic selection of N-tuple networks with consideration of inter-influence for game 2048. In *Technologies and Applications of Artificial Intelligence (TAAI 2016)*. IEEE, 2016.
- [15] Kiminori Matsuzaki. Developing a 2048 player with backward temporal coherence learning and restart. In *Advances in Computer Games (ACG 2017)*, pages 176–187. Springer, 2017.
- [16] Kiminori Matsuzaki and Madoka Teramura. A further investigation of neural network players for game 2048. In *Advances in Computer Games (ACG 2019)*. Springer, 2019.
- [17] Rahul Mehta. 2048 is (PSPACE) hard, but sometimes easy. arXiv:1408.6315, 2014.
- [18] Alok Menghrajani. 2048 with undo. <https://github.com/alokmengahrajani/2048>, 2014.
- [19] Kazuto Oka and Kiminori Matsuzaki. Systematic selection of N-tuple networks for 2048. In *Computers and Games (CG 2016)*. Springer, 2016.
- [20] Michael S. Paterson and Carl E. Hewitt. Comparative schematology. In *Record of the Project MAC Conference on Concurrent Systems and Parallel Computation*, pages 119–127, 1970.
- [21] A. J. Richardson. 2048-AI. <https://github.com/aj-r/2048-AI>, 2014.
- [22] Ravi Sethi and Jeffrey D. Ullman. The generation of optimal code for arithmetic expressions. *Journal of the ACM*, 17(4):715–728, 1970.
- [23] Alexey Slizkov. Computational bounds for the 2048 game. arXiv:2303.07266, 2023.
- [24] Marcin Szubert and Wojciech Jaśkowski. Temporal difference learning of N-tuple networks for the game 2048. In *IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–8, 2014.
- [25] Dom the Developer. 2048 Superintelligence. <https://github.com/DomTheDeveloper/2048-undo>, 2026.
- [26] I-Chen Wu, Kun-Hao Yeh, Chao-Chin Liang, Chia-Chuan Chang, and Han Chiang. Multi-stage temporal difference learning for 2048. In *Technologies and Applications of Artificial Intelligence (TAAI 2014)*, Lecture Notes in Computer Science. Springer, 2014. doi:10.1007/978-3-319-13987-6_34.

- [27] Robert Xiao. 2048-ai: an expectimax AI for 2048. <https://github.com/nneonneo/2048-ai>, 2014.
- [28] Shuhei Yamashita, Tomoyuki Kaneko, and Tomoyuki Nakayashiki. Strongly solving 2048 on 3×3 board and performance evaluation of reinforcement learning agents. In *Game Programming Workshop*. IPSJ, 2022. In Japanese.